

# 🌀 3F (FUTURO FORBES) – MASTER PROMPT BOOK  
## Sistema Completo de Finanzas Personales  
### Guía definitiva de desarrollo por fases con IA multi-modelo

> **\*\*Versión:\*\*** 2.0 (Replanteamiento desde cero)  
> **\*\*Fecha:\*\*** Febrero 2026  
> **\*\*Stack IA:\*\*** Claude (Anthropic) + Gemini (Google) + Kimi K2  
(OpenCode/Antigravity) + MiniMax M2.5 (opencode/antigravity)  
> **\*\*Plataformas objetivo:\*\*** Desktop (SQLite) | Web (PostgreSQL/MySQL) | Móvil  
(funciones core)

---

## 🏗️ ARQUITECTURA DE REFERENCIA (antes de empezar)

...

FUENTES DE INSPIRACIÓN:

|                          |                                                  |
|--------------------------|--------------------------------------------------|
| └─ MMEX (moneymanagerex) | → Core financiero, modelos de datos, UX          |
| └─ Firefly III           | → Reglas if-then, controllers single-action, API |
| REST                     |                                                  |
| └─ GnuCash               | → Double-entry, KVP metadata slots, Books        |
| └─ AdminLTE              | → UI framework base                              |
| └─ PrestaShop            | → Sistema de plugins/módulos dinámicos           |

**\*\*Principio rector:\*\*** El sistema es un **\*\*MMEX con alma web\*\***, arquitectura de plugins de PrestaShop, reglas de Firefly, contabilidad de GnuCash y cara de AdminLTE.

---

## 📁 ÍNDICE DE FASES

| Fase | Nombre                                   | Modelos IA       | Prioridad |
|------|------------------------------------------|------------------|-----------|
| 0    | Arquitectura & Diseño Base               | Claude + Gemini  | CRÍTICA   |
| 1    | Core Engine – Base de Datos & Modelos    | Kimi K2 + Claude | CRÍTICA   |
| 2    | Backend API – FastAPI Core               | Claude + Kimi K2 | CRÍTICA   |
| 3    | Sistema de Plugins (PrestaShop-style)    | Claude           | ALTA      |
| 4    | Frontend Desktop (AdminLTE + Neon)       | Gemini + Claude  | ALTA      |
| 5    | Frontend Web (Alpine.js + GridStack)     | Gemini + Claude  | ALTA      |
| 6    | Módulo Cuentas & Transacciones           | Kimi K2          | ALTA      |
| 7    | Módulo Presupuestos & Metas              | Kimi K2          | ALTA      |
| 8    | Módulo Inversiones & Activos             | Kimi K2 + Claude | ALTA      |
| 9    | Inteligencia Artificial (OCR, Forecast)  | Gemini + Claude  | MEDIA     |
| 10   | Reportes & Dashboard Analytics           | Gemini + Claude  | MEDIA     |
| 11   | Plugins Core (Dólar, Cripto, Backup)     | Kimi K2          | MEDIA     |
| 12   | Versión Móvil (PWA + React Native)       | Claude + Gemini  | MEDIA     |
| 13   | Multi-DB (SQLite desktop / PG+MySQL web) | Kimi K2 + Claude | ALTA      |
| 14   | Seguridad, Auditoría & GDPR              | Claude           | ALTA      |
| 15   | Testing E2E & CI/CD                      | Kimi K2          | MEDIA     |
| 16   | Temas & Estilos (AdminLTE + 3F Neon)     | Gemini           | MEDIA     |
| 17   | Documentación & API Spec                 | Claude           | BAJA      |
| 18   | Producción: Docker, Deploy, Monitoreo    | Kimi K2          | MEDIA     |

---

## ## 🌀 CÓMO USAR ESTE DOCUMENTO

### ### Flujo de trabajo con AntigraVity:

```

1. Abrir AntigraVity
2. Seleccionar modelo según columna "Modelos IA"
3. Copiar el prompt de la fase
4. Incluir contexto: "Tienes acceso a C:\xampp\htdocs\3F y C:\xampp\htdocs\finanzas personales"
5. Ejecutar – el agente leerá los skills relevantes automáticamente
6. Si el agente se queda sin tokens → guardar chat → continuar en nueva sesión

```

### ### Convención de skills a activar:

- `fastapi-pro` → prompts de backend
- `database-architect` → prompts de DB
- `frontend-design` + `ui-ux-designer` → prompts de UI
- `plugin-architecture` → prompts de plugins
- `python-pro` → código Python general

---

---

```
# =====  
# FASE 0: ARQUITECTURA Y DISEÑO BASE  
# Modelo: Claude (pensamiento profundo) + Gemini (investigación)  
# =====
```

### ## PROMPT 0.1 – Análisis de Referencia y Síntesis Arquitectural

**\*\*Modelo:\*\* Claude**

**\*\*Skills:\*\* `architect-review`, `senior-architect`, `architecture`**

```

Eres el arquitecto principal del sistema 3F (Futuro Forbes), un sistema de finanzas personales avanzado.

#### CONTEXTO DEL PROYECTO:

El sistema ya tiene una versión funcional en C:\xampp\htdocs\3F. Necesitamos repensar la arquitectura para hacerla definitiva, escalable y completa.

#### FUENTES DE REFERENCIA (ya analizadas, en C:\xampp\htdocs\finanzas personales):

1. MoneyManagerEX (MMEX): modelo de datos financieros, conceptos de cuentas/transacciones
2. Firefly III: arquitectura Laravel, single-action controllers, sistema de reglas if-then
3. GnuCash: double-entry accounting, KVP metadata slots, Books/Collections
4. AdminLTE: UI/UX framework
5. PrestaShop: sistema de plugins dinámicos con hooks

#### ESTADO ACTUAL DEL SISTEMA (en C:\xampp\htdocs\3F):

- Backend: FastAPI + SQLAlchemy + Python
- Frontend: Alpine.js + Jinja2 + Bootstrap 5 + AdminLTE 4
- Base de datos: MySQL/PostgreSQL/SQLite (multi-DB)
- IA: Google Gemini (OCR + Forecasting)
- Plugins: telegram\_bot, email\_smtp, dolar\_hoy, backup\_automatico, criptoya\_multi
- ~25,000 líneas de código Python, 38+ modelos SQL

#### TU TAREA:

1. Lee los archivos en C:\xampp\htdocs\3F\INFORME\_RELEVAMIENTO\_3F.md y C:\xampp\htdocs\3F\SUMMARY\_SISTEMA\_TOTAL.md
2. Lee C:\xampp\htdocs\agent\system\_blueprint.md
3. Produce un documento arquitectural definitivo que incluya:
  - a) Diagrama de arquitectura final (usar formato mermaid)
  - b) Decisiones de diseño con justificación
  - c) Mapa de módulos y dependencias
  - d) Estrategia multi-plataforma (desktop/web/mobile)
  - e) Plan de migración del código existente hacia la arquitectura definitiva
  - f) Principios de diseño no negociables
  - g) Convenciones de código para todo el equipo IA

OUTPUT: Archivo C:\xampp\htdocs\3F\docs\ARQUITECTURA\_DEFINITIVA.md  
 ```

---

```
## PROMPT 0.2 – Diseño del Schema de Base de Datos Definitivo
**Modelo:** Kimi K2 (via OpenCode)
**Skills:** `database-architect`, `database-design`, `sql-pro`
```

```

Eres el DBA principal del sistema 3F. Tu tarea es diseñar el schema de base de datos definitivo y completo.

#### CONTEXTO:

Lee todos los modelos actuales en C:\xampp\htdocs\3F\backend\models\  
 Los archivos relevantes son:

- models.py (modelos core)
- models\_advanced.py
- models\_extended.py
- models\_wealth.py
- models\_plugins.py
- models\_notifications.py
- models\_audit.py
- models\_config.py

#### INSPIRACIÓN:

- MMEX: C:\xampp\htdocs\finanzas personales\moneymanagerex
- Firefly III: C:\xampp\htdocs\finanzas personales\firefly-iii
- GnuCash: enfoque double-entry con tabla de splits

#### REQUISITOS DEL SCHEMA:

1. Soporte para SQLite (desktop), PostgreSQL y MySQL (web) sin cambiar código
2. Double-entry accounting (cada transacción tiene splits en cuentas)

3. KVP Metadata Slots (campos personalizados sin cambiar schema)
4. Árbol jerárquico de cuentas (cuenta padre/hijo recursivo)
5. Sistema de categorías con presupuestos asociados
6. Tablas de auditoría inmutables
7. Soft delete en entidades principales
8. Multi-usuario con ownership y permisos
9. Sistema de plugins con tablas dinámicas
10. Historial de precios para inversiones
11. Gestión de divisas con historial de tasas

#### ENTREGA:

Crea el archivo C:\xampp\htdocs\3F\docs\DATABASE\_SCHEMA\_DEFINITIVO.sql con:

- CREATE TABLE statements compatibles con los 3 motores
- Índices optimizados
- Foreign keys con ON DELETE estratégico
- Comentarios explicativos en cada tabla
- Un diagrama ERD en formato mermaid al final

También crea C:\xampp\htdocs\3F\backend\models\models\_v2.py con los modelos SQLAlchemy actualizados.

```

---

## PROMPT 0.3 – Definición del Sistema de Hooks y Eventos

**\*\*Modelo:\*\*** Claude

**\*\*Skills:\*\*** `plugin-architecture` (o usar `architecture-patterns`)

```

Diseña el sistema de hooks y eventos para el sistema de plugins de 3F.

#### INSPIRACIÓN:

- PrestaShop hooks: C:\xampp\htdocs\finanzas personales (repo de referencia)
- El sistema actual en: C:\xampp\htdocs\3F\backend\plugins\base.py

#### HOOKS REQUERIDOS (mínimo):

Core financiero:

- actionBeforeCreateTransaction / actionAfterCreateTransaction
- actionBeforeUpdateTransaction / actionAfterUpdateTransaction
- actionBeforeDeleteTransaction / actionAfterDeleteTransaction
- actionReconcileAccount / actionCreateBudgetAlert
- actionGoalReached / actionBudgetExceeded

Sistema:

- actionDailySummary / actionWeeklySummary / actionMonthlySummary
- actionUserLogin / actionUserLogout
- actionSystemHealthCheck
- actionBeforeExport / actionAfterExport

Display (UI injection):

- displayDashboardTop / displayDashboardBottom
- displayTransactionForm / displayAccountForm
- displayHeader / displayFooter / displaySidebar
- displayReportExtra

Datos:

- actionSyncExternalData / actionUpdateExchangeRates
- actionImportComplete / actionOCRComplete

ENTREGA:

Archivo C:\xampp\htdocs\3F\backend\core\hooks\_engine.py con:

1. Clase HooksEngine completa
2. Decoradores @hook\_listener y @hook\_dispatch
3. Sistema de prioridad para múltiples listeners
4. Async support para hooks no bloqueantes
5. Registro automático de hooks al cargar plugins
6. Sistema de tests de hooks

También documenta en C:\xampp\htdocs\3F\docs\HOOKS\_REFERENCE.md

```

---

---

```
# =====  
# FASE 1: CORE ENGINE – BASE DE DATOS & MODELOS  
# Modelo: Kimi K2 (producción masiva de código) + Claude (revisión)  
# =====
```

```
## PROMPT 1.1 – Modelos SQLAlchemy Definitivos (Completo)  
**Modelo:** Kimi K2  
**Skills:** `python-pro`, `fastapi-pro`, `database-design`
```

```

Reescribe completamente todos los modelos del sistema 3F desde cero, tomando como base los existentes y mejorándolos.

LEE PRIMERO:

1. C:\xampp\htdocs\3F\backend\models\models.py
2. C:\xampp\htdocs\3F\backend\models\models\_advanced.py
3. C:\xampp\htdocs\3F\backend\models\models\_extended.py
4. C:\xampp\htdocs\3F\backend\models\models\_wealth.py
5. C:\xampp\htdocs\3F\docs\DATABASE\_SCHEMA\_DEFINITIVO.sql (del prompt 0.2)

MEJORAS REQUERIDAS:

1. Soporte multi-DB: usar TypeDecorator para tipos compatibles con SQLite/PG/MySQL
2. Double-entry: agregar modelo TransactionSplit separado
3. KVP Metadata: agregar modelo CustomField y CustomFieldValue
4. Árbol de cuentas: account\_path como Materialized Path para queries eficientes
5. Soft delete universal: deleted\_at en todas las entidades principales
6. Auditoría automática: mixin AuditMixin con created\_at, updated\_at, created\_by\_id
7. Validaciones Pydantic en todos los schemas
8. Índices en campos frecuentes: user\_id, date, account\_id, category\_id

MODELOS A CREAR (archivos separados por dominio):

- models/user.py (User, UserPreferences, UserSession)
- models/account.py (Account, AccountType, AccountBalance, Institution)
- models/transaction.py (Transaction, TransactionSplit, TransactionTag, RecurringTransaction)
- models/category.py (Category, CategoryRule)
- models/budget.py (Budget, BudgetPeriod, BudgetLine)
- models/payee.py (Payee, PayeeRule)
- models/tag.py (Tag)
- models/investment.py (Investment, InvestmentTransaction, PriceHistory)
- models/asset.py (Asset, AssetValuation)
- models/goal.py (SavingGoal, GoalContribution)
- models/currency.py (Currency, ExchangeRate)
- models/attachment.py (Attachment, VaultDocument)
- models/notification.py (Notification, NotificationRule)
- models/audit.py (AuditLog, SecurityEvent)
- models/config.py (SystemConfig, UserConfig, ThemeConfig, LayoutConfig)
- models/plugin.py (Plugin, PluginConfig, PluginHook)
- models/custom\_field.py (CustomField, CustomFieldValue)
- models/report.py (SavedReport, ReportSchedule)

OUTPUT: Todos los archivos en C:\xampp\htdocs\3F\backend\models\  
 Actualizar C:\xampp\htdocs\3F\backend\models\\_\_init\_\_.py con todos los imports.  
 ...

---

## PROMPT 1.2 – Motor de Double-Entry Accounting  
 \*\*Modelo:\*\* Claude  
 \*\*Skills:\*\* `backend-dev-guidelines`, `python-pro`

...

Implementa el motor de contabilidad de doble entrada (double-entry accounting) para el sistema 3F.

#### CONTEXTO:

En contabilidad de doble entrada, cada transacción afecta al menos DOS cuentas.

Ejemplo: Compra de supermercado por \$1000

DEBE: Gastos/Alimentación +\$1000

HABER: Banco/Cuenta corriente -\$1000

#### REFERENCIA:

Estudia el motor de GnuCash en C:\xampp\htdocs\finanzas personales\gnucash (carpeta libgnucash/engine/ y los archivos Transaction.cpp, Split.cpp)

#### IMPLEMENTAR:

Archivo: C:\xampp\htdocs\3F\backend\core\ledger\_engine.py

#### Clases:

##### 1. LedgerEngine

- create\_transaction(splits: list[Split]) → Transaction
- validate\_balanced(splits) → bool (suma debe ser 0)
- post\_transaction(transaction\_id) → bool
- void\_transaction(transaction\_id, reason) → Transaction
- get\_account\_balance(account\_id, date\_from, date\_to) → Decimal

- get\_account\_register(account\_id, \*\*filters) → list[Entry]
- reconcile\_account(account\_id, statement\_balance, statement\_date)

## 2. Split (dataclass)

- account\_id: int
- amount: Decimal (positivo = DEBE, negativo = HABER)
- memo: str
- reconciled: bool

## 3. DoubleEntryValidator

- check\_balance(splits) → ValidationResult
- check\_accounts\_exist(splits) → ValidationResult
- check\_no\_circular\_transfer(splits) → ValidationResult

## REGLAS:

- Toda transacción DEBE estar balanceada (suma de splits = 0)
- Usar Decimal para TODOS los montos (nunca float)
- Soporte para multi-moneda (split en divisa + monto base en moneda base)
- Transacciones en estado: DRAFT → POSTED → RECONCILED → VOIDED
- Audit trail inmutable de todos los cambios

TESTS en C:\xampp\htdocs\3F\backend\tests\test\_ledger\_engine.py

```

---

## PROMPT 1.3 – Sistema de Migración Multi-DB

\*\*Modelo:\*\* Kimi K2

\*\*Skills:\*\* `database-migrations-sql-migrations`, `python-pro`

```

Implementa el sistema de migraciones de base de datos compatible con SQLite, PostgreSQL y MySQL.

## CONTEXTO DEL SISTEMA 3F:

- Versión Desktop: SQLite (un archivo .db por usuario)
- Versión Web: PostgreSQL (producción) o MySQL (alternativo)
- No usar Alembic por limitaciones con multi-DB

## HERRAMIENTAS A USAR:

- yoyo-migrations (simple, compatible con los 3 motores)
- O implementar sistema propio basado en tabla migration\_log

## IMPLEMENTAR:

### 1. C:\xampp\htdocs\3F\backend\core\migration\_manager.py

- Clase MigrationManager
- detect\_db\_type(url) → 'sqlite' | 'postgresql' | 'mysql'
- run\_migrations(db\_url) → MigrationResult
- rollback(migration\_id) → bool
- get\_current\_version() → str
- check\_pending() → list[Migration]

### 2. C:\xampp\htdocs\3F\backend\migrations\

- 001\_initial\_schema.py

- 002\_add\_audit\_tables.py
- 003\_add\_plugin\_tables.py
- 004\_add\_investment\_tables.py
- 005\_add\_custom\_fields.py
- (una migración por módulo)

3. Cada migración tiene:

- up(engine, dialect) → None (aplicar)
- down(engine, dialect) → None (revertir)
- SQL diferenciado por dialecto cuando necesario

4. C:\xampp\htdocs\3F\backend\scripts\migrate.py

- Script de línea de comandos
- Comando: python migrate.py up / down / status / reset

COMPATIBILIDAD:

- SQLite: sin ALTER TABLE complejos, usar recreación de tablas
- PostgreSQL: aprovechar JSONB para campos dinámicos
- MySQL: usar TINYINT para boolean, TEXT para JSON

...

---

---

```
# =====
# FASE 2: BACKEND API – FASTAPI CORE
# Modelo: Claude + Kimi K2
# =====
```

## PROMPT 2.1 – Estructura Base del Backend FastAPI

\*\*Modelo:\*\* Claude

\*\*Skills:\*\* `fastapi-pro`, `fastapi-templates`, `api-design-principles`

...

Rees estructura el backend FastAPI del sistema 3F con las mejores prácticas.

LEE EL BACKEND ACTUAL:

C:\xampp\htdocs\3F\backend\main.py

C:\xampp\htdocs\3F\backend\api\ (todos los archivos)

C:\xampp\htdocs\3F\backend\core\ (todos los archivos)

NUEVA ESTRUCTURA A IMPLEMENTAR:

```
backend/
├── main.py                # App factory, middleware, startup
├── config.py              # Configuración centralizada con Pydantic
Settings
├── dependencies.py        # Dependencias globales FastAPI
├── api/
│   └── v1/
│       ├── router.py      # Router principal v1
│       ├── auth.py
│       ├── accounts.py
│       └── transactions.py
```



```

|
|
| categories.py
| budgets.py
| payees.py
| tags.py
| investments.py
| assets.py
| goals.py
| reports.py
| currencies.py
| notifications.py
| vault.py
| plugins.py
| config.py
| ia.py
| audit.py
| recurring.py
| health.py
|
| v2/                                # Preparado para futuro
|
| core/
|   | security.py                    # JWT, bcrypt, permisos
|   | database.py                   # Engine, sessions, multi-DB factory
|   | ledger_engine.py              # Motor double-entry (fase 1.2)
|   | hooks_engine.py              # Sistema de hooks (fase 0.3)
|   | scheduler.py                 # Tareas programadas (APScheduler)
|   | cache.py                     # Cache en memoria / Redis opcional
|   | pagination.py                # Paginación estándar
|   | filtering.py                 # Sistema de filtros dinámicos
|   | audit_service.py             # Servicio de auditoría
|   | notification_service.py
|   | fx_service.py                # Tipos de cambio
|   | report_service.py            # Generación de reportes
|   | middleware.py                # Security headers, CORS, rate limit
|
| services/                         # Lógica de negocio por dominio

```

#### IMPLEMENTAR EN MAIN.PY:

- App factory con lifespan (startup/shutdown)
- Middleware stack completo
- Registro automático de routers v1
- Registro automático de plugins activos
- OpenAPI schema personalizado
- Health check en startup

#### ENTREGAR:

main.py + dependencies.py + config.py + api/v1/router.py

Con todos los imports correctos y comentarios de qué va en cada archivo.

```

---

## PROMPT 2.2 – API de Transacciones con Double-Entry

**\*\*Modelo:\*\*** Kimi K2

**\*\*Skills:\*\*** `fastapi-pro`, `python-pro`, `api-design-principles`

```

Implementa el endpoint completo de transacciones usando el motor double-entry.

#### REFERENCIA:

- Motor: C:\xampp\htdocs\3F\backend\core\ledger\_engine.py (creado en fase 1.2)
- Modelos: C:\xampp\htdocs\3F\backend\models\transaction.py
- API actual: C:\xampp\htdocs\3F\backend\api\transactions.py

ENDPOINTS A IMPLEMENTAR (C:\xampp\htdocs\3F\backend\api\v1\transactions.py):

#### # CRUD básico

|        |                           |                                          |
|--------|---------------------------|------------------------------------------|
| GET    | /api/v1/transactions      | # Listado paginado con filtros avanzados |
| POST   | /api/v1/transactions      | # Crear (simple o con splits)            |
| GET    | /api/v1/transactions/{id} | # Detalle con splits                     |
| PUT    | /api/v1/transactions/{id} | # Actualizar                             |
| DELETE | /api/v1/transactions/{id} | # Soft delete                            |

#### # Acciones especiales

|      |                                     |                                                   |
|------|-------------------------------------|---------------------------------------------------|
| POST | /api/v1/transactions/{id}/void      | # Anular (double-entry reversal)                  |
| POST | /api/v1/transactions/{id}/reconcile | # Marcar conciliada                               |
| POST | /api/v1/transactions/bulk           | # Creación masiva                                 |
| POST | /api/v1/transactions/split          | # Crear con splits múltiples                      |
| POST | /api/v1/transactions/transfer       | # Transferencia entre cuentas (auto double-entry) |
| POST | /api/v1/transactions/import         | # Importar desde CSV/OFX/QIF                      |
| GET  | /api/v1/transactions/duplicates     | # Detectar posibles duplicados                    |
| GET  | /api/v1/transactions/search         | # Búsqueda full-text                              |

#### # Por entidad relacionada

|     |                                      |                         |
|-----|--------------------------------------|-------------------------|
| GET | /api/v1/accounts/{id}/transactions   | # Libro mayor de cuenta |
| GET | /api/v1/categories/{id}/transactions |                         |
| GET | /api/v1/payees/{id}/transactions     |                         |

#### SCHEMAS requeridos:

- TransactionCreate (simple + con splits)
- TransactionUpdate (parcial, PATCH)
- TransactionResponse (con splits expandidos, relaciones)
- TransactionFilter (filtros: dateFrom, dateTo, accounts[], categories[], types[], status, amountMin, amountMax, search)
- TransactionImport (formato CSV con mapping de columnas)
- BulkTransactionCreate
- TransferCreate (de cuenta, a cuenta, monto, fecha, memo)
- SplitCreate / SplitResponse

#### REQUERIMIENTOS:

- Cada operación DEBE pasar por el LedgerEngine
  - Fire hooks: actionAfterCreateTransaction, etc.
  - Paginación estándar (page, page\_size, total, items)
  - Ordenamiento por cualquier campo
  - Filtros combinables con AND/OR
  - Transacciones atómicas (todo o nada)
  - Respuestas con código de error descriptivo
- ...

---

## PROMPT 2.3 – Sistema de Reglas Automáticas (Firefly-style)

**\*\*Modelo:\*\*** Claude

**\*\*Skills:\*\*** `backend-dev-guidelines`, `python-pro`

...

Implementa el sistema de reglas automáticas inspirado en Firefly III.

REFERENCIA Firefly III:

C:\xampp\htdocs\finanzas personales\firefly-iii\app\Rules\

Estudia: Trigger, Action, RuleGroup, RuleProcessor

IMPLEMENTAR: C:\xampp\htdocs\3F\backend\core\rules\_engine.py

TRIGGERS disponibles (condiciones IF):

- transaction\_amount\_is / transaction\_amount\_between
- transaction\_description\_contains / starts\_with / ends\_with / is\_regex
- transaction\_type\_is (income/expense/transfer)
- from\_account\_is / to\_account\_is
- category\_is / payee\_is / tag\_is
- transaction\_date\_on / before / after
- has\_attachment / no\_attachment
- note\_contains

ACTIONS disponibles (acciones THEN):

- set\_category(category\_id)
- set\_subcategory(subcategory\_id)
- set\_payee(payee\_id) / create\_payee(name)
- set\_tag(tag\_id) / add\_tag / remove\_tag
- set\_budget(budget\_id)
- set\_description(text) / append\_description(text)
- set\_note(text)
- convert\_currency(to\_currency)
- set\_reconciled(true/false)
- send\_notification(message)
- link\_bill(bill\_id)
- set\_custom\_field(field\_name, value)

MODELOS:

- RuleGroup (orden de ejecución, activo/inactivo)
- Rule (nombre, descripción, trigger\_mode: ALL/ANY, stop\_processing)
- RuleTrigger (tipo, operador, valor)
- RuleAction (tipo, valor, orden)

IMPLEMENTAR:

1. Clase RulesEngine

- apply\_rules(transaction) → RulesResult
- test\_rule(rule\_id, transaction) → TestResult (sin guardar)
- import\_rules(json\_export) → list[Rule]
- export\_rules() → dict

2. Endpoints: C:\xampp\htdocs\3F\backend\api\v1\rules.py

- CRUD de RuleGroups y Rules

- POST /rules/test (probar regla con transacción de ejemplo)
- POST /rules/apply-all (re-aplicar todas las reglas a transacciones históricas)
- GET /rules/suggestions (sugerir reglas basadas en patrones históricos)

3. Hook: La regla se ejecuta en `actionAfterCreateTransaction` automáticamente

---

---

```
# =====
# FASE 3: SISTEMA DE PLUGINS
# Modelo: Claude
# =====
```

## PROMPT 3.1 – Plugin Engine Completo (PrestaShop-style)

**\*\*Modelo:\*\*** Claude

**\*\*Skills:\*\*** `architecture-patterns`, `python-pro`

---

Implementa el sistema de plugins definitivo para 3F, inspirado en PrestaShop.

REFERENCIA PrestaShop:

C:\xampp\htdocs\finanzas personales (busca los repos de referencia PrestaShop)

Archivos clave de PrestaShop: `Module.php`, `Hook.php`, `ModuleManager.php`

BASE ACTUAL:

C:\xampp\htdocs\3F\backend\plugins\base.py

C:\xampp\htdocs\3F\backend\plugins\README.md

Plugins existentes: `telegram_bot`, `email_smtp`, `dolar_hoy`, `backup_automatiko`, `criptoya_multi`

ESTRUCTURA DE UN PLUGIN 3F:

```
plugins/
├── mi_plugin/
│   ├── __init__.py
│   ├── plugin.json          # Metadatos: nombre, versión, autor, hooks que
escucha
│   ├── plugin.py            # Clase principal hereda de BasePlugin
│   ├── models.py            # Modelos de BD propios (opcional)
│   ├── api.py                # Endpoints FastAPI propios (opcional)
│   ├── schemas.py           # Schemas Pydantic
│   ├── services.py           # Lógica de negocio
│   ├── migrations/           # Migraciones de BD del plugin
│   ├── templates/            # Templates HTML propios (opcional)
│   ├── static/               # Assets CSS/JS propios (opcional)
│   ├── tests/                # Tests del plugin
│   └── README.md             # Documentación
```

CLASE BASE PLUGIN (C:\xampp\htdocs\3F\backend\plugins\base.py):

```
class BasePlugin:
    name: str
```

```

version: str
author: str
description: str
hooks: list[str] # Hooks que escucha este plugin

def install(self) → bool          # Crear tablas, configs por defecto
def uninstall(self) → bool        # Eliminar tablas y datos
def activate(self) → bool
def deactivate(self) → bool
def get_config_schema() → dict # JSON Schema de configuración
def get_config() → dict
def set_config(data: dict) → bool
def render_config_page() → str # HTML para panel admin

# Hooks automáticamente descubiertos por nombre:
def hook_actionAfterCreateTransaction(self, transaction, **kwargs)
def hook_displayDashboardTop(self, **kwargs) → str # Retorna HTML

```

PLUGIN MANAGER (C:\xampp\htdocs\3F\backend\core\plugin\_manager.py):

```

- discover_plugins() → list[PluginInfo]
- install_plugin(plugin_id) → Result
- uninstall_plugin(plugin_id) → Result
- activate_plugin(plugin_id) → Result
- deactivate_plugin(plugin_id) → Result
- get_active_plugins() → list[BasePlugin]
- dispatch_hook(hook_name, **kwargs) → list[HookResult]
- install_from_zip(zip_file) → Result
- check_updates() → list[PluginUpdate]
- get_plugin_config(plugin_id) → dict
- set_plugin_config(plugin_id, data) → Result

```

API ENDPOINTS (C:\xampp\htdocs\3F\backend\api\v1\plugins.py):

```

- GET /plugins (listar con estado)
- POST /plugins/install (subir ZIP o dar nombre)
- POST /plugins/{id}/activate
- POST /plugins/{id}/deactivate
- DELETE /plugins/{id}
- GET /plugins/{id}/config
- PUT /plugins/{id}/config
- GET /plugins/{id}/logs
- POST /plugins/marketplace/search (futuro marketplace)

```

SEGURIDAD:

```

- Plugins en sandbox (no pueden acceder fuera de su directorio)
- Validación de plugin.json antes de instalar
- Sistema de permisos: qué tablas y endpoints puede acceder un plugin
- Logs de actividad por plugin
` ``

```

---

## PROMPT 3.2 – Refactorizar Plugins Existentes al Nuevo Sistema

**\*\*Modelo:\*\*** Kimi K2

**\*\*Skills:\*\*** `python-pro`, `code-refactoring-refactor-clean`

...

Migra todos los plugins existentes al nuevo sistema de plugins definido en el PROMPT 3.1.

PLUGINS A MIGRAR (en C:\xampp\htdocs\3F\backend\plugins\):

1. telegram\_bot/
2. email\_smtp/
3. dolar\_hoy/
4. backup\_automatico/
5. criptoya\_multi/
6. argentina\_datos/ (si existe)
7. crypto\_tracker/ (si existe)
8. cuentas\_wallet/ (si existe)
9. export\_tools/ (si existe)
10. ia\_forecasting/ (si existe)
11. ia\_ocr/ (si existe)

PARA CADA PLUGIN:

1. Crear plugin.json con metadatos completos
2. Refactorizar la clase principal para heredar correctamente de BasePlugin
3. Implementar install() / uninstall() / activate() / deactivate()
4. Agregar get\_config\_schema() con JSON Schema
5. Migrar los hooks al naming convention hookNombreDelHook
6. Asegurar que las migraciones de BD están en migrations/
7. Actualizar tests

TAMBIÉN CREAR NUEVOS PLUGINS:

- webhook\_generic/ (Zapier/Make/n8n integration)
- slack\_discord/ (notificaciones)
- open\_banking/ (estructura base para futuro)
- excel\_import/ (importación avanzada de Excel)
- qif\_ofx\_import/ (formatos MMEX/GnuCash)
- price\_updater/ (actualización automática de precios de acciones)
- budgets\_alerts/ (alertas de presupuesto avanzadas)
- recurring\_processor/ (procesador de transacciones recurrentes)

...

---

---

```
# =====  
# FASE 4 & 5: FRONTEND (DESKTOP + WEB)  
# Modelo: Gemini (diseño visual) + Claude (lógica)  
# =====
```

## PROMPT 4.1 – Sistema de Temas y Estilos

**\*\*Modelo:\*\*** Gemini

**\*\*Skills:\*\*** `frontend-design`, `ui-ux-designer`, `theme-factory`

...

Implementa el sistema de temas completo para 3F.

## CONTEXTO:

El sistema actual usa AdminLTE 4 Beta + CSS Neon/Cyberpunk personalizado.

Ubicación del frontend: C:\xampp\htdocs\3F\frontend\

## TEMAS A IMPLEMENTAR:

1. AdminLTE Classic (el original de AdminLTE – OBLIGATORIO incluir sin modificar)
2. 3F Neon (el tema Cyberpunk/HUD actual – mejorar)
3. 3F Dark Pro (dark moderno, tipo Notion/Linear)
4. 3F Light Clean (minimalista, tipo Firefly III)
5. 3F Solarized (colores suaves, productividad)

## ESTRUCTURA:

```
frontend/  
└─ static/  
    └─ themes/  
        └─ adminlte-classic/  
            └─ theme.css  
            └─ theme.json # Metadatos del tema  
        └─ 3f-neon/  
            └─ theme.css  
            └─ theme.json  
        └─ 3f-dark-pro/  
        └─ 3f-light-clean/  
        └─ 3f-solarized/  
    └─ css/  
        └─ base.css # Variables CSS globales
```

PARA CADA TEMA (theme.json):

```
{  
  "id": "3f-neon",  
  "name": "3F Neon",  
  "description": "Cyberpunk HUD style",  
  "preview_image": "preview.png",  
  "author": "3F Team",  
  "dark_mode": true,  
  "css_variables": {  
    "--primary": "#00ff88",  
    "--background": "#0a0a1a",  
    ...  
  }  
}
```

TEMA 3F NEON (mejorar el actual):

- Mantener el estilo Cyberpunk/HUD
- Agregar: glowing borders, scanlines effect, animated gradients
- Dashboard: KPIs con efecto neon glow
- Gráficos: colores neon sobre fondo oscuro
- Tablas: fondo semi-transparente, borde neon
- Sidebar: efecto vidrio (glassmorphism)
- Animaciones: smooth transitions, hover effects

## ENTREGA:

- 5 archivos theme.css completos

- 5 archivos theme.json
  - Sistema de CSS custom properties para cambio dinámico
  - JavaScript para cambio de tema sin reload: applyTheme(themeId)
  - Preview de cada tema en /settings/themes
- ...

---

## PROMPT 4.2 – Dashboard Modular con GridStack

**\*\*Modelo:\*\*** Gemini

**\*\*Skills:\*\*** `frontend-design`, `ui-ux-designer`

...

Diseña e implementa el dashboard principal del sistema 3F.

ESTADO ACTUAL:

C:\xampp\htdocs\3F\frontend\templates\index.html

C:\xampp\htdocs\3F\frontend\static\js\main.js

TECNOLOGÍAS:

- Alpine.js 3.x (reactividad)
- GridStack.js (drag & drop)
- Chart.js 4.x (gráficos)
- Font Awesome 6 (iconos)
- Tema activo (del sistema de temas)

WIDGETS DISPONIBLES (cada uno es un componente autónomo):

Financieros:

- widget-balance-total (balance consolidado de todas las cuentas)
- widget-monthly-summary (ingresos vs gastos del mes)
- widget-budget-progress (progress bars por categoría de presupuesto)
- widget-recent-transactions (últimas 5-10 transacciones)
- widget-accounts-list (lista de cuentas con saldo)
- widget-cashflow-chart (gráfico de flujo de caja 12 meses)
- widget-category-donut (distribución de gastos por categoría)
- widget-goals-progress (estado de metas de ahorro)
- widget-upcoming-recurring (próximas transacciones programadas)

Inversiones:

- widget-portfolio-value (valor total del portafolio)
- widget-stock-tickers (precios en tiempo real via WebSocket)
- widget-portfolio-distribution (distribución por tipo de inversión)
- widget-portfolio-performance (rendimiento %)

IA & Analytics:

- widget-ia-insights (sugerencias de IA)
- widget-spending-heatmap (heatmap de gastos)
- widget-forecast-mini (mini pronóstico próximos 30 días)

Cotizaciones (Plugins):

- widget-dolar-hoy (cotizaciones dólar Argentina)
- widget-crypto-prices (precios cripto)

Sistema:



- widget-notifications (notificaciones recientes)
- widget-quick-add (agregar transacción rápida)
- widget-calendar (vista calendario de transacciones)

#### COMPORTAMIENTO:

- Layout guardado en BD por usuario (GridStack serialization)
- Botón "Editar Dashboard": activa modo drag & drop
- Panel lateral "Agregar Widget": listado de widgets disponibles
- Cada widget: botón para configurar, minimizar, eliminar
- Responsive: en mobile el grid pasa a lista vertical
- Auto-refresh: datos actualizables sin reload
- Loading skeleton: mientras carga datos

#### IMPLEMENTAR:

frontend/templates/index.html (completo, con Alpine.js state management)  
 frontend/static/js/dashboard.js (lógica de widgets y GridStack)  
 frontend/static/js/widgets/ (un .js por widget)  
 ```

---

#### ## PROMPT 4.3 – Formularios de Transacciones (UX Premium)

**\*\*Modelo:\*\*** Gemini + Claude

**\*\*Skills:\*\*** `frontend-design`, `ui-ux-designer`

```

Implementa el formulario de transacciones con UX de primera clase.

#### REFERENCIA MMEX:

Estudia la UX de MMEX en C:\xampp\htdocs\finanzas personales\moneymanagerex

#### TIPOS DE FORMULARIO:

1. Transacción Simple (gasto/ingreso)
2. Transferencia entre cuentas
3. Transacción con Splits (dividir en múltiples categorías)
4. Recurrente (configurar repetición)
5. Quick-Add (modal rápido, mínimo campos)

#### CAMPOS Y UX:

- Fecha: datepicker con atajos (hoy, ayer, esta semana)
- Tipo: toggle visual (GASTO/INGRESO/TRANSFERENCIA)
- Monto: input grande, formato automático, calculadora integrada
- Cuenta: dropdown con saldo actual visible, búsqueda
- Categoría: hierarchical picker con búsqueda, colores
- Beneficiario/Payee: autocomplete con creación inline
- Descripción: autocomplete basado en historial
- Tags: multiple select con colores
- Estado: toggle reconciliado/pendiente
- Notas: expandible
- Adjuntos: drag & drop de archivos/fotos
- OCR: botón "Escanear Ticket" → sube foto → IA extrae datos

#### FUNCIONALIDADES ESPECIALES:

- Split Mode: agregar filas con monto+categoría (suma debe coincidir con total)

- Calculadora:  $150+30*2 \rightarrow$  auto-evalúa y pone resultado
- Auto-completar: si escribo "McDonald"  $\rightarrow$  pone categoría y payee automático (regla)
- Duplicar transacción
- Crear transacción recurrente desde existente
- Batch edit: seleccionar múltiples y cambiar categoría/cuenta

#### IMPLEMENTAR:

frontend/templates/transactions/ (carpeta con todos los templates)  
 frontend/static/js/transaction-form.js (lógica completa)  
 frontend/static/js/ocr-handler.js (integración con API de OCR)

ESTILO: Usar el tema activo, modal o página full, responsive

```

---

---

```
# =====
# FASE 6, 7, 8: MÓDULOS FINANCIEROS CORE
# Modelo: Kimi K2
# =====
```

```
## PROMPT 6.1 – Módulo Completo de Cuentas
**Modelo:** Kimi K2
**Skills:** `fastapi-pro`, `frontend-developer`
```

```

Implementa el módulo de cuentas completo (backend + frontend).

TIPOS DE CUENTA (inspirado en MMEX + GnuCash):

- CHECKING (cuenta corriente)
- SAVINGS (caja de ahorro)
- CASH (efectivo)
- CREDIT\_CARD (tarjeta de crédito)
- INVESTMENT (cuenta inversiones)
- LOAN (préstamo)
- TERM\_DEPOSIT (plazo fijo)
- CRYPTO\_WALLET (billetera cripto)
- OTHER

BACKEND (C:\xampp\htdocs\3F\backend\api\v1\accounts.py):

Endpoints:

|        |                                         |                                 |
|--------|-----------------------------------------|---------------------------------|
| GET    | /api/v1/accounts                        | # Todas las cuentas del usuario |
| POST   | /api/v1/accounts                        | # Crear cuenta                  |
| GET    | /api/v1/accounts/{id}                   | # Detalle                       |
| PUT    | /api/v1/accounts/{id}                   | # Actualizar                    |
| DELETE | /api/v1/accounts/{id}                   | # Soft delete                   |
| GET    | /api/v1/accounts/{id}/balance           | # Saldo actual (calculado)      |
| GET    | /api/v1/accounts/{id}/balance/history   | # Historial de saldos           |
| GET    | /api/v1/accounts/{id}/transactions      | # Libro mayor                   |
| POST   | /api/v1/accounts/{id}/reconcile         | # Iniciar conciliación          |
| POST   | /api/v1/accounts/{id}/reconcile/confirm | # Confirmar conciliación        |

```
GET    /api/v1/accounts/summary          # Resumen de todas las cuentas
GET    /api/v1/accounts/net-worth        # Patrimonio neto (activos - pasivos)
```

CONCILIACIÓN (feature clave de MMEX):

- El usuario ingresa el saldo del extracto bancario
- El sistema lista las transacciones pendientes de conciliar
- El usuario las marca como conciliadas
- Al confirmar, guarda el punto de conciliación

FRONTEND (C:\xampp\htdocs\3F\frontend\templates\accounts\):

- index.html: Lista de cuentas con tarjetas visuales por tipo
- detail.html: Libro mayor de la cuenta (register)
- create.html / edit.html: Formulario de cuenta
- reconcile.html: Interfaz de conciliación
- Incluir: saldo, gráfico de tendencia, últimas transacciones

```

---

## PROMPT 7.1 – Módulo Completo de Presupuestos

**\*\*Modelo:\*\*** Kimi K2

**\*\*Skills:\*\*** `fastapi-pro`, `frontend-developer`

```

Implementa el módulo de presupuestos completo.

CONCEPTOS:

- Budget: presupuesto mensual/anual con categorías asignadas
- BudgetLine: categoría + monto asignado + monto gastado
- Budget puede ser: MENSUAL (se resetea), ANUAL, ROLLING (ventana deslizante)

BACKEND (C:\xampp\htdocs\3F\backend\api\v1\budgets.py):

```
GET    /api/v1/budgets
POST   /api/v1/budgets
GET    /api/v1/budgets/{id}
PUT    /api/v1/budgets/{id}
DELETE /api/v1/budgets/{id}
GET    /api/v1/budgets/{id}/status          # Estado actual (gastado vs
asignado)
GET    /api/v1/budgets/{id}/report          # Reporte detallado por período
GET    /api/v1/budgets/{id}/projections     # Proyección de fin de período
POST   /api/v1/budgets/{id}/copy           # Copiar presupuesto a nuevo período
GET    /api/v1/budgets/current              # Presupuesto activo del mes actual
```

LÓGICA DE ALERTAS:

- Si gasto supera X% del presupuesto → dispara hook actionBudgetAlertThreshold
- Si gasto supera 100% → dispara hook actionBudgetExceeded
- Enviar notificación in-app + email (si plugin email activo)

FRONTEND:

- Vista mensual: barras de progreso por categoría (verde/amarillo/rojo)
- Vista anual: tabla con cada mes como columna
- Vista comparativa: este mes vs mes anterior
- Wizard para crear presupuesto desde cero o copiar del mes anterior

- Drag & drop para asignar montos por categoría  
```

---

## PROMPT 8.1 – Módulo de Inversiones y Activos

\*\*Modelo:\*\* Kimi K2 + Claude

\*\*Skills:\*\* `fastapi-pro`, `data-scientist`

```

Implementa el módulo de inversiones y activos.

TIPOS DE INVERSIÓN:

- STOCK (acciones)
- ETF / FUND (fondos)
- BOND (bonos)
- CRYPTO (criptomonedas)
- TERM\_DEPOSIT (plazo fijo con tasa)
- REAL\_ESTATE (inmuebles como inversión)
- OTHER

BACKEND INVERSIONES:

|                                           |                                   |
|-------------------------------------------|-----------------------------------|
| GET /api/v1/investments                   | # Portfolio completo              |
| POST /api/v1/investments                  | # Agregar posición                |
| GET /api/v1/investments/{id}              |                                   |
| PUT /api/v1/investments/{id}              |                                   |
| DELETE /api/v1/investments/{id}           |                                   |
| POST /api/v1/investments/{id}/transaction | # Comprar/vender más              |
| GET /api/v1/investments/{id}/history      | # Historial de precios            |
| GET /api/v1/investments/performance       | # Rendimiento del portafolio      |
| GET /api/v1/investments/allocation        | # Distribución por tipo           |
| POST /api/v1/investments/update-prices    | # Actualizar precios (via plugin) |

BACKEND ACTIVOS:

|                                      |                         |
|--------------------------------------|-------------------------|
| GET /api/v1/assets                   |                         |
| POST /api/v1/assets                  |                         |
| GET /api/v1/assets/{id}              |                         |
| PUT /api/v1/assets/{id}              |                         |
| DELETE /api/v1/assets/{id}           |                         |
| POST /api/v1/assets/{id}/revalue     | # Revaluar activo       |
| GET /api/v1/assets/{id}/depreciation | # Calcular depreciación |

CÁLCULOS FINANCIEROS (usar Decimal siempre):

- Valor de mercado actual del portafolio
- Ganancia/pérdida realizada vs no realizada
- Rendimiento porcentual (con y sin dividendos)
- IRR (Internal Rate of Return) si hay múltiples compras
- Depreciación: lineal, doble saldo decreciente, suma de dígitos
- ROI por período

FRONTEND:

- Dashboard de portafolio: valor total, rendimiento, distribución (donut)
- Tabla de posiciones con colores rojo/verde por P&L
- Gráfico de evolución del portafolio en el tiempo

- Simulador de escenarios (si precio sube X%, ¿cuánto gano?)  
```

---

---

```
# =====  
# FASE 9: INTELIGENCIA ARTIFICIAL  
# Modelo: Gemini (OCR/Forecast) + Claude (lógica)  
# =====
```

```
## PROMPT 9.1 – Motor IA Multi-Modelo  
**Modelo:** Claude  
**Skills:** `ai-engineer`, `llm-application-dev-ai-assistant`
```

```

Implementa el motor de IA multi-modelo para 3F.

MODELOS A INTEGRAR:

1. Google Gemini 1.5 Flash (OCR de tickets, análisis de imagen)
2. Google Gemini 1.5 Pro (forecasting, análisis complejo)
3. Claude API (sugerencias de categorización, insights financieros)
4. Modo offline: reglas heurísticas cuando no hay API key

ARCHIVO: C:\xampp\htdocs\3F\backend\core\ia\_engine.py

FUNCIONES A IMPLEMENTAR:

1. OCR de Tickets (Gemini Vision):  
analyze\_receipt(image\_bytes) → ReceiptData:
  - store\_name, date, total, subtotal, tax
  - items: list[{description, quantity, unit\_price, total}]
  - payment\_method (si visible)
  - Confianza de cada campo extraído
2. Categorización Automática (Claude o Gemini):  
suggest\_category(description, amount, payee) → CategorySuggestion:
  - category\_id, subcategory\_id
  - confidence: float (0-1)
  - reasoning: str
3. Forecasting Financiero (Gemini Pro):  
forecast\_cashflow(user\_id, months\_ahead) → ForecastResult:
  - predicted\_income\_by\_month: list[Decimal]
  - predicted\_expenses\_by\_month: list[Decimal]
  - predicted\_balance: list[Decimal]
  - anomalies\_detected: list[Anomaly]
  - recommendations: list[str]

Algoritmo: usar histórico de 12+ meses + transacciones recurrentes conocidas + estacionalidad (gastos navideños, vacaciones, etc.)

4. Detección de Anomalías:

```
detect_anomalies(transactions) → list[Anomaly]:  
- Gasto inusualmente alto en categoría  
- Transacción duplicada posible  
- Patrón de fraude potencial
```

5. Insights Financieros Personalizados (Claude):  
generate\_insights(user\_id, period) → list[Insight]:  
- tipo: warning / tip / achievement / forecast  
- título y descripción  
- acción sugerida (con deeplink al módulo relevante)

#### CONFIGURACIÓN:

- Cada función tiene fallback si la API no responde
- Rate limiting para evitar costos inesperados
- Cache de respuestas para evitar re-procesar
- Logs de uso y costos de API

ENDPOINTS: C:\xampp\htdocs\3F\backend\api\v1\ia.py

```
POST /api/v1/ia/ocr  
POST /api/v1/ia/suggest-category  
GET /api/v1/ia/forecast  
GET /api/v1/ia/insights  
POST /api/v1/ia/detect-duplicates  
```\n
```

---

---

```
# =====  
# FASE 10: REPORTES & ANALYTICS  
# Modelo: Gemini (visualizaciones) + Claude (SQL)  
# =====
```

```
## PROMPT 10.1 – Motor de Reportes Completo  
**Modelo:** Claude  
**Skills:** `data-storytelling`, `sql-optimization-patterns`  
```\n
```

Implementa el motor de reportes completo del sistema 3F.

#### REFERENCIA MMEX:

C:\xampp\htdocs\finanzas personales\moneymanagerex (ver módulo de reportes)

#### REPORTES ESTÁNDAR A IMPLEMENTAR:

1. Flujo de Caja (cashflow.html)
  - Ingresos vs Egresos por período (mensual/trimestral/anual)
  - Gráfico de barras comparativo
  - Tabla detallada por categoría
  - Exportar PDF/Excel
2. Distribución por Categoría (categories.html)
  - Donut chart + tabla

- Drill-down: click en categoría → ver subcategorías y transacciones
- Comparar período actual vs anterior

### 3. Tendencias (trends.html)

- Líneas de tendencia por categoría en el tiempo
- Media móvil
- Predicción para próximos meses

### 4. Heatmap de Gastos (heatmap.html)

- Día de la semana vs hora del día
- Mes vs día del mes
- Identifica cuándo gastas más

### 5. Patrimonio Neto (net-worth.html)

- Evolución del patrimonio en el tiempo
- Activos vs pasivos
- Desglose por tipo de cuenta

### 6. Inversiones (investments-report.html)

- Rendimiento del portafolio vs benchmark
- Dividendos recibidos
- Ganancias/pérdidas realizadas por año fiscal

### 7. Presupuesto vs Real (budget-report.html)

- Cada categoría: asignado vs gastado vs año anterior
- Semáforo visual (verde/amarillo/rojo)
- Proyección de fin de período

### 8. Reporte Personalizado (custom-report.html)

- Editor visual: seleccionar dimensiones y métricas
- Guardar como reporte favorito
- Programar envío automático por email

#### IMPLEMENTAR:

C:\xampp\htdocs\3F\backend\core\report\_service.py

- Clase ReportService con método por reporte
- Queries SQL optimizadas para cada reporte
- Soporte para filtros: dateFrom, dateTo, accounts[], categories[]
- Caché de reportes pesados (invalidar cuando hay nuevas transacciones)

C:\xampp\htdocs\3F\backend\api\v1\reports.py

- GET /api/v1/reports/{report\_type}?filters...
- POST /api/v1/reports/export (generar PDF/Excel)
- GET /api/v1/reports/saved (reportes guardados del usuario)
- POST /api/v1/reports/saved (guardar reporte)
- GET /api/v1/reports/dashboard (datos para el dashboard)

```

---

---

# =====

# FASE 11: PLUGINS CORE

# Modelo: Kimi K2

# =====

## PROMPT 11.1 – Plugin: Cotizaciones Argentina & Latinoamérica

\*\*Modelo:\*\* Kimi K2

\*\*Skills:\*\* `python-pro`

...

Mejora y unifica los plugins de cotizaciones para Argentina y Latinoamérica.

PLUGINS A UNIFICAR/MEJORAR:

- dolar\_hoy/ (cotizaciones dólar Argentina)
- criptoya\_multi/ (cripto en 11 países)
- argentina\_datos/ (si existe)

NUEVO PLUGIN UNIFICADO: cotizaciones\_latam/

FUENTES DE DATOS:

Argentina:

- bluelytics.com.ar → dólar blue
- ambito.com → dólar MEP, CCL
- criptoya.com → tasas cripto

Multi-país (via CriptoYa):

- /api/{coin}/{fiat}/{volumen} → mejor precio por país

DATOS A OBTENER:

- USD Blue, MEP, CCL, Tarjeta, Oficial (Argentina)
- BTC, ETH, USDT, USDC en ARS, BRL, CLP, COP, MXN, etc.
- Actualización automática cada N minutos (configurable)
- Alertas si el tipo de cambio supera umbral configurado

WIDGETS:

- displayDashboardTop: barra con cotizaciones en tiempo real
- Página dedicada: /cotizaciones con histórico y gráficos

HOOKS QUE DISPARA:

- actionDailySummary: incluir resumen de cotizaciones del día

...

---

## PROMPT 11.2 – Plugin: Importador de Formatos MMEX/GnuCash/Firefly

\*\*Modelo:\*\* Kimi K2

\*\*Skills:\*\* `python-pro`, `data-engineer`

...

Implementa el plugin importador de múltiples formatos.

PLUGIN: importador\_universal/

FORMATOS A SOPORTAR:

1. MMEX SQLite (.mmb files):

- Conectar a la BD SQLite de MMEX



- Mapear Account → Account, Transaction → Transaction
- Preservar categorías, beneficiarios, notas
- Wizard visual de mapeo de campos

## 2. GnuCash XML (.gnucash files):

- Parsear XML de GnuCash
- Mapear Chart of Accounts a estructura 3F
- Preservar splits (ya que GnuCash también es double-entry)
- Importar tags y notas

## 3. Firefly III (API o JSON export):

- Import via API de Firefly III
- Import via JSON export de Firefly

## 4. CSV genérico:

- Wizard de mapeo de columnas
- Detección automática de formato de fecha y número
- Preview antes de confirmar importación
- Guardar perfiles de importación (para reimportar el mismo banco)

## 5. OFX/QFX (estándar bancario):

- Importar extractos bancarios
- Auto-match con transacciones existentes para conciliación

## 6. Excel/XLSX:

- Plantillas de importación predefinidas
- Detección automática de estructura

## ENDPOINTS:

```
POST /api/v1/import/preview    # Preview sin guardar (muestra N registros)
POST /api/v1/import/execute    # Ejecutar importación
GET  /api/v1/import/history    # Historial de importaciones
POST /api/v1/import/rollback/{import_id} # Deshacer importación
...
```

---

---

```
# =====
# FASE 12: VERSIÓN MÓVIL (PWA + React Native base)
# Modelo: Claude + Gemini
# =====
```

```
## PROMPT 12.1 – Progressive Web App (PWA)
**Modelo:** Gemini
**Skills:** `mobile-developer`, `frontend-design`
...
```

Convierte el frontend de 3F en una Progressive Web App completa.

## FUNCIONALIDADES PWA:

1. Service Worker para cache offline
2. Web App Manifest para instalación en home screen

3. Push notifications (Web Push API)
4. Background sync para transacciones offline
5. Share target (recibir fotos de tickets para OCR)

FUNCIONES MÓVILES CORE (no todas las del desktop):

- Dashboard resumido (balance, últimas transacciones)
- Agregar transacción rápida (quick-add optimizado para mobile)
- Ver lista de cuentas y saldos
- Escanear ticket con cámara → OCR automático
- Notificaciones push de alertas de presupuesto
- Ver reportes básicos (cashflow, categorías)
- Configuración básica

EXCLUIR DEL MÓVIL:

- Dashboard drag & drop (GridStack)
- Reportes complejos
- Gestión de plugins
- Importación masiva
- Configuración avanzada del sistema

IMPLEMENTAR:

frontend/static/js/service-worker.js

frontend/static/manifest.json

frontend/templates/mobile/ (templates optimizados para móvil)

frontend/static/css/mobile.css (touch-friendly, bottom navigation, etc.)

DISEÑO MÓVIL:

- Bottom navigation bar (5 items: Home, Transactions, Add, Reports, Settings)
- Cards grandes con touch targets de 44px mínimo
- Swipe actions en transacciones (swipe left → eliminar, right → editar)
- Pull to refresh
- Gestos nativos
- Dark mode automático según sistema

...

---

## PROMPT 12.2 – API Endpoints Mobile-Optimized

**\*\*Modelo:\*\* Claude**

**\*\*Skills:\*\* `api-design-principles`, `fastapi-pro`**

...

Crea endpoints optimizados para la versión móvil.

PRINCIPIOS:

- Respuestas compactas (menos campos que la API web)
- Endpoint de "home data" que retorna todo de una vez (evitar múltiples requests)
- Compresión gzip habilitada
- Paginación infinita (cursor-based, no page-based)
- Soporte para modo offline con sync posterior

ENDPOINTS MOBILE:

GET /api/v1/mobile/home # Balance total + últimas 5 transacciones +

```

alertas
GET  /api/v1/mobile/dashboard      # Datos del dashboard simplificado
POST /api/v1/mobile/quick-add      # Agregar transacción (mínimos campos)
POST /api/v1/mobile/ocr            # Subir foto de ticket
GET  /api/v1/mobile/accounts       # Cuentas con saldo (campos mínimos)
GET  /api/v1/mobile/transactions   # Listado optimizado para infinite scroll
GET  /api/v1/mobile/summary        # Resumen del mes (ingresos, gastos, balance)

```

SYNC OFFLINE:

```

POST /api/v1/mobile/sync          # Enviar transacciones creadas offline
  - Recibe lista de transacciones con timestamps
  - Detecta conflictos
  - Retorna estado de cada ítem (created/conflict/error)
...

```

---

---

```

# =====
# FASE 13: MULTI-DB (SQLite / PostgreSQL / MySQL)
# Modelo: Kimi K2 + Claude
# =====

```

## PROMPT 13.1 – Abstracción Multi-DB

**\*\*Modelo:\*\*** Kimi K2

**\*\*Skills:\*\*** `database-architect`, `python-pro`

...

Implementa la capa de abstracción para soportar los tres motores de BD.

CONTEXTO:

- Desktop (versión standalone): SQLite, un archivo .db por usuario/empresa
- Web compartido: PostgreSQL (recomendado)
- Web alternativo: MySQL/MariaDB
- El código no debe cambiar entre motores

IMPLEMENTAR: C:\xampp\htdocs\3F\backend\core\database.py

```
class DatabaseManager:
```

```
    def get_engine(db_type: str, connection_string: str) → Engine
```

```
    def get_session_factory(engine) → sessionmaker
```

```
class SQLiteConfig:
```

```
    # Para desktop: un archivo por empresa/usuario
```

```
    file_path: Path
```

```
    enable_wal: bool = True # Write-Ahead Logging para performance
```

```
    pragma_settings: dict # PRAGMA journal_mode, cache_size, etc.
```

```
class PostgreSQLConfig:
```

```
    host, port, database, user, password
```

```
    pool_size: int = 10
```

```
    max_overflow: int = 20
```

```
    use_ssl: bool
```

```
class MySQLConfig:
    host, port, database, user, password
    charset: str = 'utf8mb4'
    use_ssl: bool
```

#### COMPATIBILIDAD DE TIPOS:

Crear TypeDecorators para:

- JSON: JSONB en PG, JSON en MySQL, TEXT en SQLite
- Boolean: nativo en PG/MySQL, INTEGER(1) en SQLite
- Decimal: NUMERIC en PG/MySQL, REAL en SQLite (con precisión forzada)
- DateTime con timezone: TIMESTAMPTZ en PG, DATETIME en MySQL, TEXT ISO8601 en SQLite
- UUID: UUID nativo en PG, CHAR(36) en MySQL/SQLite

#### MULTI-TENANT (para versión web):

- Estrategia: cada usuario tiene su propio schema en PG (database: 3f\_main, schema: user\_123)
- Para SQLite: un archivo .db por usuario

#### CONEXIÓN DESKTOP:

- Abrir archivo .mmb de MMEX directamente (compatibilidad)
  - O archivo .3fb propio
  - Recent files list
  - Cifrado de archivo con SQLCipher (opcional, para privacidad)
- ```
...
```

---

---

```
# =====
# FASE 14: SEGURIDAD, AUDITORÍA & GDPR
# Modelo: Claude
# =====
```

## PROMPT 14.1 – Sistema de Seguridad Completo

\*\*Modelo:\*\* Claude

\*\*Skills:\*\* `security-auditor`, `auth-implementation-patterns`,  
`gdpr-data-handling`

...

Implementa el sistema de seguridad completo para 3F.

#### LEER PRIMERO:

C:\xampp\htdocs\3F\backend\core\security.py (actual)

C:\xampp\htdocs\3F\backend\core\security\_middlewares.py (actual)

#### IMPLEMENTAR / MEJORAR:

##### 1. AUTENTICACIÓN:

- JWT con refresh tokens (access: 15min, refresh: 7 días)
- 2FA opcional (TOTP con Google Authenticator)
- "Remember me" con long-lived tokens

- Detección de login desde nueva IP → notificación email
- Bloqueo tras N intentos fallidos
- Rate limiting en endpoints auth (5 req/min)

## 2. AUTORIZACIÓN:

- RBAC: roles USER, ADMIN, SUPERUSER
- Por defecto, usuario solo ve SUS datos
- Compartir acceso entre usuarios (cuentas familiares)
- Permisos de plugins (qué puede hacer cada plugin)

## 3. MIDDLEWARE DE SEGURIDAD (mejorar el actual):

- Security Headers: HSTS, CSP, X-Frame-Options, X-Content-Type-Options
- CORS configurado estrictamente
- Request ID para trazabilidad
- Sanitización de inputs (prevenir XSS, SQLi)
- Rate limiting global y por endpoint

## 4. GDPR / PRIVACIDAD:

- Derecho al olvido: DELETE /api/v1/users/me/delete-all (elimina todos los datos del usuario permanentemente)
- Portabilidad: GET /api/v1/users/me/export (exporta TODOS los datos en JSON/CSV)
- Consentimiento: registro de cuándo el usuario aceptó los términos
- Logs de acceso a datos PII

## 5. AUDITORÍA COMPLETA:

- Loggear TODO: CREATE, READ (datos sensibles), UPDATE, DELETE
- Tabla audit\_log: inmutable (solo INSERT, nunca UPDATE/DELETE)
- Campos: user\_id, action, entity\_type, entity\_id, old\_values, new\_values, ip, timestamp
- Alertas automáticas para patrones sospechosos
- Reporte de auditoría exportable

## 6. CIFRADO:

- Datos sensibles en Vault: cifrado AES-256
- Contraseñas: bcrypt con cost factor 12
- Tokens: firmar con secret + user fingerprint
- Datos en tránsito: forzar HTTPS en producción

## ENDPOINTS DE SEGURIDAD:

|                                   |                                |
|-----------------------------------|--------------------------------|
| GET /api/v1/auth/sessions         | # Sesiones activas del usuario |
| DELETE /api/v1/auth/sessions/{id} | # Revocar sesión               |
| POST /api/v1/auth/2fa/setup       | # Configurar 2FA               |
| POST /api/v1/auth/2fa/verify      | # Verificar código 2FA         |
| GET /api/v1/users/me/audit-log    | # Actividad del usuario        |
| GET /api/v1/users/me/export       | # Exportar mis datos (GDPR)    |
| DELETE /api/v1/users/me           | # Eliminar mi cuenta (GDPR)    |

```

---

---

# =====

```
# FASE 15: TESTING E2E & CI/CD
# Modelo: Kimi K2
# =====
```

```
## PROMPT 15.1 – Suite de Tests Completa
```

```
**Modelo:** Kimi K2
```

```
**Skills:** `python-testing-patterns`, `playwright-skill`, `tdd-workflow`
```

```
```
```

Implementa la suite de tests completa para el sistema 3F.

LEER PRIMERO:

C:\xampp\htdocs\3F\backend\tests\ (tests actuales)

ESTRATEGIA:

1. Unit Tests (pytest): funciones aisladas, servicios, cálculos
2. Integration Tests (pytest + TestClient): endpoints API con BD test
3. E2E Tests (Playwright): flujos completos en el browser

UNIT TESTS (C:\xampp\htdocs\3F\backend\tests\unit\):

- test\_ledger\_engine.py: validaciones double-entry
- test\_rules\_engine.py: reglas automáticas
- test\_report\_service.py: cálculos de reportes
- test\_currency\_converter.py: conversiones de moneda
- test\_recurring\_service.py: generación de transacciones recurrentes
- test\_plugin\_manager.py: carga y descarga de plugins

INTEGRATION TESTS (C:\xampp\htdocs\3F\backend\tests\integration\):

- test\_auth\_flow.py: registro, login, 2FA, refresh
- test\_transactions\_api.py: CRUD completo con splits
- test\_budgets\_api.py: presupuestos y alertas
- test\_investments\_api.py: portafolio y precios
- test\_import\_api.py: importación de diferentes formatos
- test\_reports\_api.py: generación de reportes

E2E TESTS (C:\xampp\htdocs\3F\frontend\tests\e2e\):

- test\_login\_flow.spec.ts
- test\_add\_transaction.spec.ts (incluye OCR mock)
- test\_dashboard\_widgets.spec.ts
- test\_budget\_management.spec.ts
- test\_theme\_switching.spec.ts

CONFIGURACIÓN:

- pytest.ini con coverage mínimo del 80%
- fixtures: base de datos de test (SQLite en memoria)
- factories: generadores de datos falsos (faker)
- mocks: APIs externas (Gemini, cotizaciones)
- CI ready: sin dependencias externas en tests

SCRIPTS:

C:\xampp\htdocs\3F\Makefile (o scripts\test.ps1 para Windows):

- make test → todos los tests
- make test-unit → solo unit
- make test-integration → solo integration

```
- make test-e2e → solo Playwright
- make coverage → reporte de cobertura HTML
```
```

---

---

```
# =====
# FASE 16: TEMAS & ESTILOS AVANZADOS
# Modelo: Gemini
# =====
```

```
## PROMPT 16.1 – Implementar Todos los Temas del Sistema
**Modelo:** Gemini
**Skills:** `frontend-design`, `ui-ux-designer`
```

```

Implementa el sistema de temas completo con todos los estilos.

CONTEXTO:

Ya existe: tema Neon en C:\xampp\htdocs\3F\frontend\static\css\  
AdminLTE está en: C:\xampp\htdocs\finanzas personales\AdminLTE4\

TAREA:

Toma el AdminLTE original sin modificar y agrégalo como primer tema disponible.  
Luego implementa los 4 temas propios de 3F:

TEMA 1: AdminLTE Classic (Original)

- Copiar CSS de AdminLTE4 sin ningún cambio
- Solo agregar theme.json con metadatos
- Sirve como referencia y fallback

TEMA 2: 3F Neon (Cyberpunk HUD) – Mejorar existente

Mejoras específicas:

- Variables CSS para cada color (--neon-green, --neon-blue, etc.)
- Efecto scanlines en el fondo (sutil, no invasivo)
- Borders con neon glow animado en cards hover
- Números financieros en font monospace con color según positivo/negativo
- Loading spinners con animación neon
- Transiciones suaves entre páginas (fade)

TEMA 3: 3F Dark Pro

Inspiración: Notion, Linear, Vercel Dashboard

- Fondo: #0d1117 (GitHub dark)
- Texto: #e6edf3
- Accent: #58a6ff (azul GitHub)
- Cards: #161b22 con border #30363d
- Sin efectos llamativos, diseño funcional y limpio
- Typography: Inter/SF Pro

TEMA 4: 3F Light Clean

Inspiración: Firefly III, Stripe Dashboard

- Fondo blanco, cards con sombra sutil

- Colores pastel para categorías
- Tipografía: Open Sans o Lato
- Data tables con rayas alternadas
- Muy legible en sol/pantallas brillantes

#### TEMA 5: 3F Solarized

Basado en la paleta Solarized (Ethan Schoonover)

- Base: #002b36 (dark) o #fdf6e3 (light)
- Dos variantes: Solarized Dark y Solarized Light
- Colores de acento de la paleta oficial Solarized

#### SISTEMA DE CAMBIO:

- /api/v1/config/theme (GET/PUT)
- Cambio instantáneo via JavaScript sin reload
- Preview en /settings con iframe del dashboard
- Guardar preferencia en BD del usuario
- Modo automático: seguir sistema operativo (prefers-color-scheme)

```

---

---

```
# =====
# FASE 17: DOCUMENTACIÓN
# Modelo: Claude
# =====
```

```
## PROMPT 17.1 – Documentación Completa
**Modelo:** Claude
**Skills:** `api-documentation-generator`, `docs-architect`
```

```

Genera la documentación completa del sistema 3F.

#### DOCUMENTACIÓN A GENERAR:

1. README.md (raíz del proyecto):
  - Descripción, capturas de pantalla, features
  - Instalación rápida (5 pasos)
  - Docker one-liner
  - Arquitectura en 2 párrafos
  - Links a documentación detallada
2. docs/INSTALL.md:
  - Instalación en Windows (XAMPP)
  - Instalación en Linux/Mac
  - Instalación con Docker
  - Configuración post-instalación
3. docs/CONFIGURATION.md:
  - Todas las variables de entorno documentadas
  - Ejemplos de .env para cada entorno
  - Configuración de BD (SQLite/PG/MySQL)



- Configuración de modelos IA

4. docs/PLUGINS.md:

- Cómo instalar un plugin
- Cómo crear un plugin (tutorial)
- Referencia de hooks
- Lista de plugins oficiales

5. docs/API.md:

- Referencia completa de endpoints
- Ejemplos de requests/responses
- Autenticación
- Rate limits

6. docs/THEMES.md:

- Cómo cambiar de tema
- Cómo crear un tema personalizado
- Variables CSS disponibles

7. docs/USER\_GUIDE.md:

- Guía de usuario para cada módulo
- Screenshots de cada pantalla
- Tips y atajos de teclado

8. Inline code documentation:

- Docstrings en todas las funciones importantes (Google style)
- Type hints completos en Python
- JSDoc en JavaScript

FORMATO: Markdown con diagramas Mermaid donde corresponda

IDIOMA: Español (con glosario técnico al final)

```

---

---

```
# =====  
# FASE 18: PRODUCCIÓN  
# Modelo: Kimi K2  
# =====
```

```
## PROMPT 18.1 – Docker y Deploy Completo  
**Modelo:** Kimi K2  
**Skills:** `docker-expert`, `deployment-engine`
```

```

Configura el ambiente de producción completo para 3F.

LEER PRIMERO:

C:\xampp\htdocs\3F\docker-compose.yml (actual)

C:\xampp\htdocs\3F\Dockerfile (actual)

IMPLEMENTAR VERSIÓN DEFINITIVA:

1. docker-compose.yml (producción):  
Servicios:
  - backend: FastAPI + Uvicorn (3 workers)
  - frontend: Nginx para servir static + proxy al backend
  - db\_postgres: PostgreSQL 16
  - db\_mysql: MySQL 8 (alternativo)
  - redis: Cache y sesiones
  - celery: Cola de tareas para OCR y reportes pesados
  - celery-beat: Tareas programadas
  - flower: Monitor de Celery
  - prometheus: Métricas
  - grafana: Dashboards de monitoreo
2. docker-compose.dev.yml (desarrollo):
  - Hot reload en backend (--reload)
  - Volúmenes para código fuente
  - adminer para BD
  - MailHog para emails de testing
3. docker-compose.desktop.yml (versión desktop local):
  - Solo backend + frontend + sqlite
  - Sin servicios externos
  - Una sola imagen auto-contenida
4. Nginx config:
  - Proxy reverso al backend
  - Servir archivos estáticos
  - Compresión gzip
  - Cache para assets (1 año)
  - Headers de seguridad
  - SSL con certbot
5. GitHub Actions (.github/workflows/):
  - ci.yml: tests en cada push
  - deploy-staging.yml: deploy a staging en PR merge
  - deploy-production.yml: deploy a producción en tag
  - docker-build.yml: build y push a registry
6. Scripts de mantenimiento (scripts/):
  - backup.sh: backup de BD + attachments
  - restore.sh: restaurar backup
  - health\_check.sh: verificar estado del sistema
  - update.sh: actualizar el sistema sin downtime

#### MONITOREO:

- Prometheus metrics endpoint: /metrics
  - Grafana dashboard: transacciones/min, latencia API, errores
  - Alertas: cuando el sistema tiene errores > 1% o latencia > 2s
  - Logs estructurados (JSON) para fácil parsing
- ...

---

---

```
# =====  
# PROMPTS DE MANTENIMIENTO Y CONTINUACIÓN  
# Para usar cuando se retoma el desarrollo  
# =====
```

```
## PROMPT CONTEXT RESTORE – Para retomar una sesión  
**Usar al inicio de cada nueva sesión en Antigravity**
```

---

RETOMANDO DESARROLLO DEL SISTEMA 3F

Estado del proyecto:

- Repositorio: C:\xampp\htdocs\3F
- Documentación: C:\xampp\htdocs\3F\docs\
- Estado general: C:\xampp\htdocs\3F\SUMMARY\_SISTEMA\_TOTAL.md
- Skills disponibles: C:\xampp\htdocs\.agent\skills\
- Plan maestro: [este archivo]

ÚLTIMA FASE COMPLETADA: [COMPLETAR]

PRÓXIMA TAREA: [COMPLETAR]

Por favor:

1. Lee C:\xampp\htdocs\3F\SUMMARY\_SISTEMA\_TOTAL.md para contexto general
  2. Lee C:\xampp\htdocs\3F\estado\_proyecto.md para el estado actual
  3. Confirma qué archivos existen en la fase que vamos a trabajar
  4. Continúa desde donde quedamos
- 

---

```
## PROMPT CHECKPOINT – Guardar estado para continuar luego  
**Usar cuando el contexto se está agotando**
```

---

GUARDANDO ESTADO DE SESIÓN

Antes de que se agoten los tokens, por favor:

1. Actualiza C:\xampp\htdocs\3F\estado\_proyecto.md con:
    - Qué implementamos en esta sesión
    - Archivos creados/modificados
    - Estado de cada componente
    - Qué falta completar
    - Próximo paso exacto
  2. Si hay código a medias, deja comentarios TODO claros en el código indicando exactamente qué falta implementar
  3. Genera un resumen de 5 bullets de lo hecho en esta sesión
  4. Indica el prompt exacto con el que retomar en la próxima sesión
-

---

## PROMPT AUDIT – Revisar calidad del código  
\*\*Usar periódicamente o al terminar una fase\*\*

```

#### AUDITORÍA DE CÓDIGO 3F

Por favor realiza una auditoría completa de:  
[ESPECIFICAR DIRECTORIO O ARCHIVOS]

Verificar:

1. Sintaxis Python válida (py\_compile en cada archivo)
2. Sin console.log ni print() de debug
3. Todos los TODO/FIXME documentados o resueltos
4. Imports usados (sin imports muertos)
5. Manejo de errores en todos los try/catch
6. Tipos correctos en todas las funciones
7. Tests para las funciones críticas
8. Docstrings en funciones públicas
9. Sin credenciales hardcodeadas
10. SQL sin inyecciones posibles

Reporta:

- Lista de problemas encontrados por severidad (CRÍTICO / MAYOR / MENOR)
  - Sugerencias de mejora
  - Estimación de cobertura de tests actual
- ```

---

## PROMPT MIGRATION\_CHECK – Verificar migración desde MMEX  
\*\*Para cuando el usuario quiera migrar datos existentes\*\*

```

#### MIGRACIÓN DE DATOS MMEX A 3F

El usuario tiene una base de datos MMEX en: [RUTA AL ARCHIVO .mmb]

Por favor:

1. Lee el schema de MMEX desde:  
C:\xampp\htdocs\finanzas personales\moneymanagerex\
2. Mapea las tablas de MMEX a tablas de 3F:
  - ACCOUNTLIST → accounts
  - CHECKINGACCOUNT → transactions
  - BUDGETTABLE → budgets
  - CATEGSPLIT → categories
  - PAYEE → payees
  - etc.
3. Crea el script de migración usando el plugin importador\_universal

4. Ejecuta en modo preview primero (--dry-run)
5. Reporta: N cuentas, N transacciones, N categorías a importar y cualquier dato que no se pueda migrar (con justificación)
6. Si el usuario confirma, ejecutar la migración real

```

---

```
# =====  
# ROADMAP FUTURO (Post v2.0)  
# =====
```

#### ## Versión 2.1 – Open Banking

```

Implementar integración con APIs bancarias:

- Open Banking (PSD2 para Europa)
- Plaid (para USA/Canadá)
- Argentina: integración con APIs de bancos locales
- Auto-sync de cuentas bancarias
- Import automático de movimientos
- Matching con transacciones existentes (anti-duplicado)

```

#### ## Versión 2.2 – Colaboración Familiar

```

Sistema multi-usuario compartido:

- Cuentas compartidas entre usuarios (familia)
- Permisos granulares (solo lectura / lectura-escritura / admin)
- Chat interno por transacción (comentarios)
- Notificaciones entre usuarios del mismo grupo
- Reportes consolidados del grupo
- Presupuesto familiar con contribuciones individuales

```

#### ## Versión 2.3 – App Móvil Nativa (React Native)

```

App nativa con características adicionales:

- Notificaciones push nativas
- Biometría (FaceID / Fingerprint)
- Widget en pantalla de inicio (saldo actual)
- NFC para pagos registrados automáticamente
- Integración con Google Pay / Apple Pay para registro
- Modo offline completo con sync inteligente

```

#### ## Versión 3.0 – IA Avanzada

```

Consejero financiero IA personalizado:

- Chat natural con el sistema ("¿Cuánto gasté en comida este mes?")
- Detección automática de suscripciones olvidadas
- Comparativa con benchmarks anónimos de usuarios similares
- Plan de ahorro automático basado en objetivos

- Predicción de flujo de caja a 6 meses
  - Alertas inteligentes de anomalías
  - Optimización automática de categorías
- ...

---

## # 📋 CHECKLIST DE IMPLEMENTACIÓN

## Por completar antes del release v2.0:

### ### Fase 0 (Arquitectura)

- [ ] ARQUITECTURA\_DEFINITIVA.md
- [ ] DATABASE\_SCHEMA\_DEFINITIVO.sql
- [ ] HOOKS\_REFERENCE.md

### ### Fase 1 (Core Engine)

- [ ] Todos los modelos SQLAlchemy v2
- [ ] ledger\_engine.py con double-entry
- [ ] migration\_manager.py

### ### Fase 2 (Backend API)

- [ ] Estructura nueva backend/api/v1/
- [ ] Endpoints de transacciones con double-entry
- [ ] Sistema de reglas automáticas

### ### Fase 3 (Plugins)

- [ ] Plugin Engine completo
- [ ] Todos los plugins migrados al nuevo sistema
- [ ] 3 nuevos plugins (webhook, importador, price\_updater)

### ### Fase 4/5 (Frontend)

- [ ] Sistema de temas (5 temas)
- [ ] Dashboard con todos los widgets
- [ ] Formularios de transacciones mejorados

### ### Fase 6/7/8 (Módulos)

- [ ] Módulo cuentas completo con conciliación
- [ ] Módulo presupuestos con proyecciones
- [ ] Módulo inversiones con cálculos financieros

### ### Fase 9 (IA)

- [ ] Motor IA multi-modelo
- [ ] OCR mejorado
- [ ] Forecasting avanzado
- [ ] Insights automáticos

### ### Fase 10 (Reportes)

- [ ] 8 reportes estándar
- [ ] Reporte personalizado
- [ ] Exportación PDF/Excel mejorada

### ### Fase 11 (Plugins Core)

- [ ] Plugin cotizaciones\_latam unificado

- ☐ Importador universal (6 formatos)

#### ### Fase 12 (Móvil)

- ☐ PWA completa
- ☐ Endpoints mobile-optimized
- ☐ Sync offline

#### ### Fase 13 (Multi-DB)

- ☐ Abstracción multi-DB completa
- ☐ SQLite desktop mode
- ☐ Multi-tenant para web

#### ### Fase 14 (Seguridad)

- ☐ 2FA implementado
- ☐ GDPR completo
- ☐ Auditoría mejorada

#### ### Fase 15 (Testing)

- ☐ 80%+ cobertura backend
- ☐ Tests E2E principales flujos

#### ### Fase 16 (Temas)

- ☐ 5 temas implementados
- ☐ AdminLTE original incluido

#### ### Fase 17 (Docs)

- ☐ README.md actualizado
- ☐ Guías de instalación
- ☐ Documentación API

#### ### Fase 18 (Producción)

- ☐ Docker compose definitivo
- ☐ CI/CD con GitHub Actions
- ☐ Monitoreo con Prometheus/Grafana

---

> **\*\*Nota final:\*\*** Este documento es el mapa completo. Cada prompt está diseñado para producir código funcional, no bocetos. Al completar todas las fases, el sistema 3F tendrá feature-parity con MMEX + Firefly III + GnuCash, más capacidades de IA que ninguno de ellos tiene.

> **\*\*Para modificaciones futuras:\*\*** Actualizar este archivo con nuevos prompts en la sección correspondiente y marcar el checklist a medida que se completan.

---

\*Generado: Febrero 2026 | Sistema: 3F Futuro Forbes v2.0\*